Jakarta Persistence Applied 온라인 부록

Oracle HR Persistence

권진

Version 1.0: Jakarta Persistence 3.2 판

Table of Contents

Oracle HR Persistence 1
oracle-sample-schemas-persistence-hr 2
HR (Human Resources) 4
Attribute Mappings 7
Exercises 24

Oracle HR Persistence

이 장에서는 Oracle 의 예제 스키마 가운데 HR(Human Resources)을 매핑한다. 회사의 지역과 나라, 사무소, 부서, 직무, 직원, 그리고 직원이 거쳐 온 직무의 이력을 담은 작은 스키마다.

모듈의 구성과 매핑의 규칙, Oracle 의 형, 테스트, 데이터베이스를 시작하는 방법은 [oracle-sample-schemas-persistence] 장에서 보았다. 먼저 HR 을 매핑한 `oracle-sample-schemas-persistence-hr` 모듈에만 있는 클래스를 살펴본다. 다음으로 HR 스키마의 테이블과 뷰, 그 관계와 HR 에서 새로 보는 것을 보고, 컬럼을 속성에 어떻게 매핑했는지 주제마다 살펴본다. 마지막으로 연습문제를 모았다.

HR 에서 새로 보는 것은 셋이다. 서로를 참조하는 두 테이블, 행을 고칠 때 다른 테이블에 행을 넣는 트리거, 매핑이 쓰지 않는 시퀀스가 그렇다.

oracle-sample-schemas-persistence-hr

`oracle-sample-schemas-persistence-hr` 는 HR 을 Jakarta Persistence 로 매핑한 모듈이다. HR 이 어떤 스키마인지는 다음 절인 HR (Human Resources) 에서 살펴보고, 이 절에서는 모듈이 어떻게 구성되어 있는지를 먼저 본다.

모듈의 구조와 `module-info.java`, 테스트의 구성은 세 모듈이 같으므로 [oracle-sample-schemas-persistence] 장에서 CO 의 모듈로 보았다. 이 절에서는 HR 의 모듈에만 있는 클래스를 본다.

메인 코드

`hr.mapped` 에는 테이블 7 개와 뷰 1 개마다 매핑된 슈퍼클래스가 하나씩, 모두 8 개 있다. `MappedRegion` 부터 `MappedEmpDetailsView` 까지, 이름은 `Mapped` 뒤에 엔티티의 이름을 붙인 것이다. 매핑된 슈퍼클래스 말고도 아래의 클래스가 있다.

Table 1. hr.mapped 패키지의 다른 클래스

클래스	하는 일
`JobHistoryId`	`JOB_HISTORY` 의 복합 기본 키. `@IdClass` 로 지정하는 보통의 클래스다.
`_Constants`, `_Utils`	여러 클래스가 함께 쓰는 상수와 메서드를 두는 패키지 전용 클래스. 프로시저 `SECURE_DML` 이 허용하는 시각과 요일, 오류 번호를 상수로 두고, 같은 판단을 자바로 하는 메서드를 둔다.

테스트 코드

테스트 코드의 `hr` 패키지에는 매핑된 슈퍼클래스를 상속한 엔티티가 8 개 있다. 엔티티의 이름은 테이블의 이름을 단수로 바꾼 것이고, 뷰의 엔티티는 뷰의 이름을 그대로 따른다. HR 에는 키가 없는 뷰가 없으므로, 뷰 `EMP_DETAILS_VIEW` 도 직원의 식별자를 식별자로 삼는 엔티티다.

Table 2. HR 의 엔티티

테이블 또는 뷰	엔티티	식별자
`REGIONS`	`Region`	`REGION_ID`
`COUNTRIES`	`Country`	`COUNTRY_ID`
`LOCATIONS`	`Location`	`LOCATION_ID`
`DEPARTMENTS`	`Department`	`DEPARTMENT_ID`
`JOBS`	`Job`	`JOB_ID`
`EMPLOYEES`	`Employee`	`EMPLOYEE_ID`
`JOB_HISTORY`	`JobHistory`	`EMPLOYEE_ID`, `START_DATE`. `@IdClass(JobHistoryId.class)`.
`EMP_DETAILS_VIEW`	`EmpDetailsView`	`EMPLOYEE_ID`

[oracle-sample-schemas-persistence-tests] 에서 본 테스트 클래스는 8 개 엔티티 모두에 있고, 퍼시스터만 `JobHistory` 와 `EmpDetailsView` 를 뺀 6 개에 있다. `JOB_HISTORY` 의 행은 트리거가 넣으므로(UPDATE_JOB_HISTORY 트리거) `JobHistory` 에도 퍼시스터가 없다.

그 밖의 테스트는 아래와 같다.

- `Employee_Static_Test` 는 `Employee` 가 선언한 컬럼의 길이와 범위의 상수가 서로 맞는지 검사한다.

- `Job_TestUtils` 는 `Job` 의 최저 급여와 최고 급여에 넣을 값을 만들고, `Job_TestUtils_Test` 가 그 값이 `Job` 이 받는 범위 안에 있는지 검사한다.
- 테스트 코드의 `hr.mapped` 패키지에는 `JobHistoryId_Test` 와 `JobHistoryId_Randomizer`, 그리고 `_Utils` 가 프로시저 `SECURE_DML` 과 같이 판단하는지 검사하는 `_Utils_Test` 가 있다.

HR (Human Resources)

HR(Human Resources)은 Oracle 이 배포하는 예제 스키마 가운데 하나로, 회사의 인사 부서를 본뜬 작은 스키마다.[1] Oracle Database 의 문서가 SQL 과 PL/SQL 의 예를 보일 때 이 스키마를 쓴다. 이 장은 2022 년에 나온 21.1 판을 쓴다.

`HR` 스키마에는 테이블 7 개와 뷰 1 개가 있다. 테이블은 크게 두 묶음으로 나눌 수 있다.

Table 3. HR 스키마의 테이블과 뷰

묶음	테이블과 뷰
조직	`REGIONS`, `COUNTRIES`, `LOCATIONS`, `DEPARTMENTS`
직원	`JOBS`, `EMPLOYEES`, `JOB_HISTORY`
뷰	`EMP_DETAILS_VIEW`

`COUNTRIES` 는 `REGIONS` 를, `LOCATIONS` 는 `COUNTRIES` 를, `DEPARTMENTS` 는 `LOCATIONS` 를 참조한다. `EMPLOYEES` 는 `JOBS` 와 `DEPARTMENTS` 를 참조하고, 자기 자신도 참조한다. `JOB_HISTORY` 는 `EMPLOYEES`, `JOBS`, `DEPARTMENTS` 를 참조한다. 두 묶음은 `DEPARTMENTS` 와 `EMPLOYEES` 에서 이어지는데, 이 두 테이블은 서로를 참조한다. `EMPLOYEES` 의 `DEPARTMENT_ID` 는 직원이 일하는 부서를, `DEPARTMENTS` 의 `MANAGER_ID` 는 부서를 맡은 직원을 참조한다. 뷰 `EMP_DETAILS_VIEW` 는 직원을 부서, 직무, 사무소, 나라, 지역과 조인해 한 행으로 보여 준다.

묶음마다 테이블과 그 사이의 관계를 그림으로 보면 아래와 같다. 그림에는 같은 묶음 안의 관계만 그렸다.

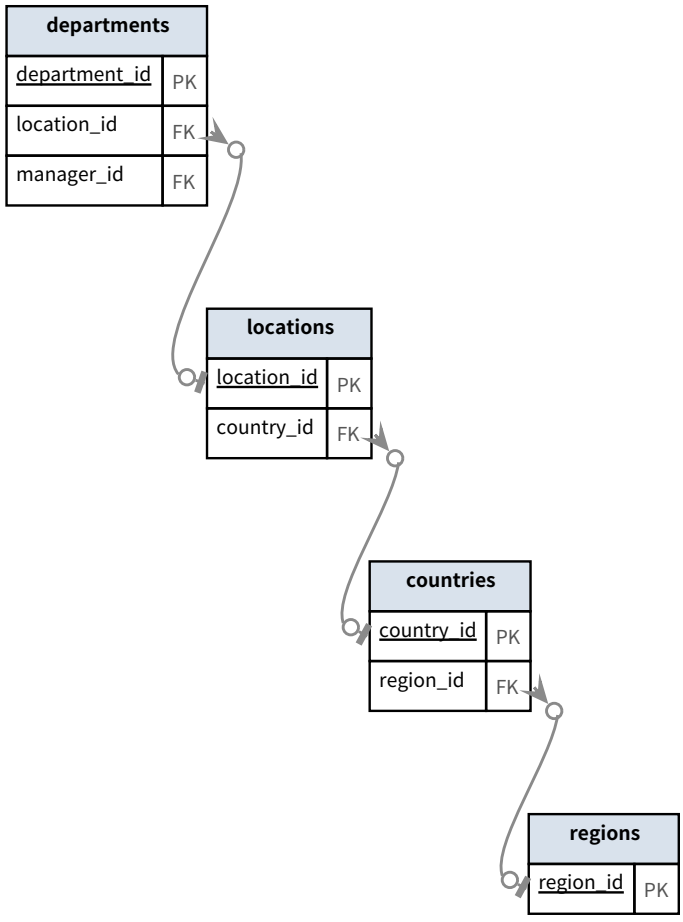


Figure 1. 조직

Table 4. 조직 묶음

테이블	담는 것	행
REGIONS	지역. 번호와 이름	5
COUNTRIES	나라. 두 글자의 코드와 이름, 속한 지역	25
LOCATIONS	회사의 사무소와 창고, 생산 시설이 있는 곳의 주소	23
DEPARTMENTS	부서. 이름과 부서장, 부서가 있는 곳	27

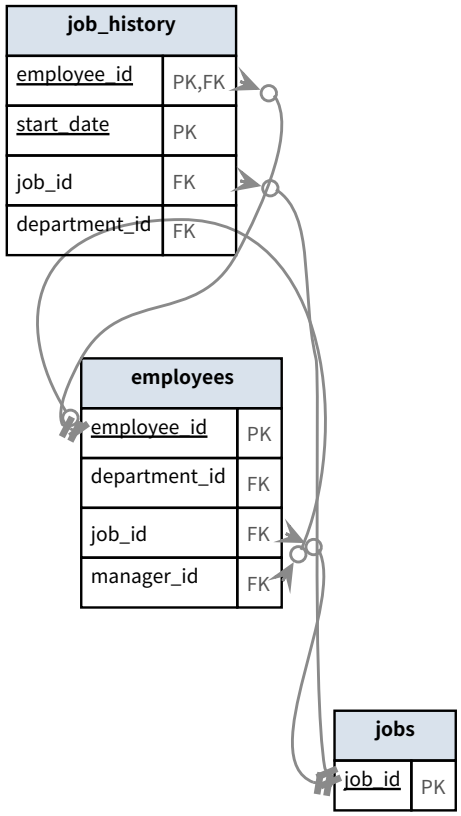


Figure 2. 직원

Table 5. 직원 묶음

테이블	담는 것	행
JOBS	직무. 직무의 이름과 급여 범위	19
EMPLOYEES	직원. 이름과 연락처, 입사일, 직무, 급여, 상사, 부서	107
JOB_HISTORY	직원이 직무나 부서를 바꾸기 전의 직무와 부서	10

HR 은 작은 스키마다. 가장 큰 EMPLOYEES 에도 107 행이 들어 있을 뿐이다.

뷰 EMP_DETAILS_VIEW 에는 106 행이 있어, EMPLOYEES 보다 하나 적다. 뷰는 직원을 부서와 e.department_id = d.department_id 로 조인하는데, 부서가 없는 직원 Kimberely Grant 는 이 조건을 만족하지 못하기 때문이다.

시퀀스와 트리거

HR 에는 테이블과 뷰 말고도 시퀀스 셋과 트리거 둘, 프로시저 둘이 있다.

Table 6. HR 스키마의 시퀀스, 트리거, 프로시저

이름	종류	하는 일
LOCATIONS_SEQ	시퀀스	3300 부터 100 씩 늘어나는 사무소의 번호.
DEPARTMENTS_SEQ	시퀀스	280 부터 10 씩 늘어나는 부서의 번호.
EMPLOYEES_SEQ	시퀀스	207 부터 1 씩 늘어나는 직원의 번호.
UPDATE_JOB_HISTORY	트리거	직원의 JOB_ID 나 DEPARTMENT_ID 를 고치면, 고치기 전의 직무와 부서를 ADD_JOB_HISTORY 로 JOB_HISTORY 에 넣는다.
SECURE_EMPLOYEES	트리거	EMPLOYEES 를 고치기 전에 SECURE_DML 을 호출한다. 설치할 때 비활성화된다.
SECURE_DML	프로시저	평일 08:00 부터 18:00 까지가 아니면 오류 -20205 를 낸다.

시퀀스의 시작 값은 설치한 데이터의 가장 큰 번호 다음이다. **그러나 이 장의 매핑은 시퀀스를 쓰지 않는다.** 어느 엔티티의 식별자에도 `@GeneratedValue` 가 없고, 식별자의 값은 영속화하기 전에 애플리케이션이 정한다.

UPDATE_JOB_HISTORY 트리거

EMPLOYEES 에는 트리거 UPDATE_JOB_HISTORY 가 걸려 있다.

UPDATE_JOB_HISTORY 트리거

```
CREATE OR REPLACE TRIGGER update_job_history
  AFTER UPDATE OF job_id, department_id ON employees
  FOR EACH ROW
BEGIN
  add_job_history(:old.employee_id, :old.hire_date, sysdate,
                  :old.job_id, :old.department_id);
END;
```

JOB_ID 나 DEPARTMENT_ID 를 고치는 UPDATE 마다, 고치기 전의 직무와 부서를 JOB_HISTORY 에 한 행으로 넣는다. 시작일은 직원의 HIRE_DATE, 종료일은 그때의 SYSDATE 다. 엔티티 Employee 의 직무나 부서를 바꾸고 플러시하면, 구현체가 보내는 UPDATE 에 트리거가 반응한다.

Hibernate ORM 은 기본으로 UPDATE 에 엔티티의 모든 컬럼을 넣는다. 그래서 입사일만 고쳐도 UPDATE 에 JOB_ID 가 들어가 트리거가 실행된다.

트리거는 HIRE_DATE 를 고치지 않는다. 그래서 같은 직원의 직무나 부서를 두 번 바꾸면 두 번째 행의 시작일이 첫 번째 행과 같아지고, JOB_HISTORY 의 기본 키를 위반한다. 직무와 부서를 함께 바꿀 때에는 두 변경을 한 번의 플러시에 담아 UPDATE 를 하나로 보내야 한다.

Attribute Mappings

HR 의 테이블 7 개에서 매핑된 슈퍼클래스가 컬럼을 매핑한 속성은 모두 35 개이고, 컬럼마다 속성이 꼭 하나씩 있다. Oracle 의 형을 어떤 자바의 형으로 매핑하는지는 [oracle-types] 에서 보았다. 이 절은 먼저 그 규칙을 HR 의 어느 컬럼에 적용했는지 요약하고, 그다음 HR 이 매핑에 더하는 것을 주제마다 살펴본다. 테이블을 하나씩 살펴보지는 않는다.

요약

- `VARCHAR2` 와 `CHAR` 는 `String` 이다(문자열). 변환기를 쓰는 속성은 없다.
- `NUMBER(p)` 는 `Integer`, `NUMBER(p,s)` 는 `BigDecimal` 이다(수). 정밀도가 없는 `NUMBER` 인 `REGION_ID` 는 `Long` 이다(정밀도가 없는 NUMBER).
- `DATE` 는 `LocalDateTime` 이다(날짜와 시각).
- 식별자에 `@GeneratedValue` 가 없다(식별자).
- 테이블이 지키지 않는 규칙은 매핑도 검사하지 않는다(테이블이 지키지 않는 규칙).
- 외래 키 컬럼은 두 번 매핑한다. 서로를 참조하는 `DEPARTMENTS` 와 `EMPLOYEES`, 자기 자신을 참조하는 `EMPLOYEES` 도 이렇게 매핑한다(서로를 참조하는 DEPARTMENTS 와 EMPLOYEES, 자기 자신을 참조하는 직원).
- 트리거가 행을 넣는 `JOB_HISTORY` 는 읽기 전용으로 매핑하고, 복합 기본 키는 `@IdClass` 인 `JobHistoryId` 로 매핑한다(트리거가 쓰는 JOB_HISTORY).
- 뷰까지 세면 속성은 51 개다. 뷰 `EMP_DETAILS_VIEW` 의 속성은 읽어 온 테이블의 속성과 형이 같다(뷰 EMP_DETAILS_VIEW).

문자열

HR 은 `VARCHAR2` 의 길이에 아무것도 붙이지 않았으므로, 길이를 바이트로 센다. 그래서 `EMPLOYEES.FIRST_NAME` 같은 컬럼에서는 `@Size` 를 통과한 값도 데이터베이스가 거부할 수 있다([oracle-types]).

`CHAR` 인 컬럼은 `COUNTRIES.COUNTRY_ID` 와 그것을 참조하는 `LOCATIONS.COUNTRY_ID` 뿐이다.

CHAR 인 나라의 코드

COUNTRIES.COUNTRY_ID 는 GB, US 같은 두 글자의 코드를 담는 CHAR(2) 이고, 테이블의 기본 키다. CHAR 는 길이가 정해진 문자열이다. 값이 길이보다 짧으면 Oracle 이 뒤를 공백으로 채우므로, 읽은 String 의 뒤에도 공백이 붙는다.

MappedCountry 의 countryId

```
// TODO: 지운다. DDL 이 제약하지 않는다 -- COUNTRIES.COUNTRY_ID 는 CHAR(2) 다. 길이로는 최소
//  길이가 정해지지 않는다
//    @Size(min = SIZE_MIN_COUNTRY_ID, max = SIZE_MAX_COUNTRY_ID)
@Size(max = SIZE_MAX_COUNTRY_ID)
@NotNull
@Id
@Basic(optional = false)
@Column(name = COLUMN_NAME_COUNTRY_ID, nullable = false, insertable = true, updatable = false,
        length = COLUMN_LENGTH_COUNTRY_ID,
        comment = "Primary key of countries table.")
private String countryId;
```

@Size 에는 max 만 있다. 코드는 늘 두 글자지만, 길이가 2 인 CHAR 컬럼도 한 글자의 값을 받는다. 그래서 매핑은 테이블이 지키지 않는 최소 길이를 검사하지 않는다(테이블이 지키지 않는 규칙). min 까지 기재했던 @Size 는 주석으로 남아 있다.

이 컬럼을 참조하는 LOCATIONS.COUNTRY_ID 도 CHAR(2) 다. MappedLocation 은 이 외래 키의 길이를 상수로 따로 둔다. 그 값이 참조하는 MappedCountry 의 것과 같은지는 정적 초기화 블록의 assert 가 확인한다.

MappedLocation 의 정적 초기화 블록

```
static {
    assert COLUMN_LENGTH_COUNTRY_ID == MappedCountry.COLUMN_LENGTH_COUNTRY_ID;
}
```

assert 는 JVM 이 단언을 켰을 때(-ea)만 검사한다. 단언을 켠 채로 두 상수가 어긋나면 MappedLocation 클래스를 초기화하다가 AssertionError 가 난다.

문자열 식별자와 빈 문자열

JOBS.JOB_ID 도 COUNTRY_ID 처럼 문자열 식별자다. AD_PRES, IT_PROG 처럼 부서와 직급을 줄여 붙인 코드이고, 시퀀스로 매기는 번호가 아니다.

MappedJob 의 jobId

```
@Size(min = SIZE_MIN_JOB_ID, max = SIZE_MAX_JOB_ID)
@NotNull
@Id
@Column(name = COLUMN_NAME_JOB_ID,
        nullable = false,
        insertable = true,
        updatable = false,
        length = COLUMN_LENGTH_JOB_ID,
        comment = "Primary key of jobs table."
)
private String jobId;
```

VARCHAR2(10) 은 공백을 채우지 않는 문자열이므로, CHAR(2) 인 COUNTRY_ID 와 달리 넣은 값을 그대로 읽는다. @Size 의 min

은 0 이다. 빈 문자열을 막지 않는 것처럼 보이지만, Oracle 은 길이가 0 인 문자열을 NULL 로 다룬다.[2] 그래서 NOT NULL 인 이 컬럼에는 빈 문자열을 넣을 수 없다.

수

HR 의 NUMBER(p) 는 모두 여섯 자리 이하이므로, EMPLOYEE_ID, DEPARTMENT_ID, LOCATION_ID 같은 식별자와 JOBS.MIN_SALARY, JOBS.MAX_SALARY 는 Integer 다. EMPLOYEES.SALARY 와 EMPLOYEES.COMMISSION_PCT 는 소수(小數, decimal fraction) 자리가 있으므로 BigDecimal 이다(급여와 수수료율).

REGIONS.REGION_ID 와 그것을 참조하는 COUNTRIES.REGION_ID 만은 정밀도가 없는 NUMBER 이고, Long 으로 매핑한다(정밀도가 없는 NUMBER).

자릿수와 범위

LOCATIONS.LOCATION_ID 는 NUMBER(4) 다. 소수점 아래 없이 네 자리까지의 정수(整數, integer)를 담으므로, 담을 수 있는 값은 -9999 부터 9999 까지다.

MappedLocation 의 LOCATION_ID 에 관한 상수

```
public static final int COLUMN_PRECISION_LOCATION_ID = 4;
public static final int COLUMN_SCALE_LOCATION_ID = 0;
public static final int COLUMN_MIN_LOCATION_ID = -9999;
public static final int COLUMN_MAX_LOCATION_ID = +9999;
public static final int ATTRIBUTE_MIN_LOCATION_ID = COLUMN_MIN_LOCATION_ID;
public static final int ATTRIBUTE_MAX_LOCATION_ID = COLUMN_MAX_LOCATION_ID;
```

COLUMN_ 으로 시작하는 상수는 컬럼이 담을 수 있는 값을, ATTRIBUTE_ 로 시작하는 상수는 속성이 받아들일 값을 나타낸다. 이 매핑에서는 둘이 같다.

MappedLocation 의 locationId

```
@Max(ATTRIBUTE_MAX_LOCATION_ID)
@Min(ATTRIBUTE_MIN_LOCATION_ID)
@NotNull
@Id
@Column(name = COLUMN_NAME_LOCATION_ID,
        nullable = false,
        insertable = true,
        updatable = false,
        precision = COLUMN_PRECISION_LOCATION_ID,
        scale = COLUMN_SCALE_LOCATION_ID,
        comment = "Primary key of locations table"
)
private Integer locationId;
```

@Column 의 precision 과 scale 은 컬럼의 자릿수를 기재할 뿐이다.[3] 영속화할 값이 그 자릿수에 들어가는지는 @Min 과 @Max 가 검사한다. DEPARTMENTS 의 DEPARTMENT_ID 도 같은 NUMBER(4) 이고 같은 범위를 검사한다. NUMBER(6) 인 EMPLOYEES 의 EMPLOYEE_ID 와 JOBS 의 MIN_SALARY, MAX_SALARY 는 -999999 부터 999999 까지를 검사한다.

정밀도가 없는 NUMBER

REGIONS.REGION_ID 는 자릿수를 정하지 않은 NUMBER 다. HR 의 다른 테이블은 식별자의 자릿수를 NUMBER(4) 나 NUMBER(6) 처럼 정하고, 매핑은 그 컬럼을 Integer 로 받는다. 형만 보면 38 자리의 수와 소수도 담을 수 있으므로 BigDecimal 이 맞지만, 이 프로젝트는 자릿수가 없는 이 컬럼을 Long 으로 받는다. 그 이유는 [oracle-types] 에서 보았다. 이 컬럼을 참조하는 COUNTRIES.REGION_ID 도 Long 이다.

MappedRegion 의 regionId

```
@NotNull
@Id
@Column(name = COLUMN_NAME_REGION_ID, nullable = false, insertable = true, updatable = false,
        comment = "Primary key of regions table.")
private Long regionId;
```

자릿수가 없으므로 값의 범위를 검사하는 @Min 이나 @Max 도 없다. updatable = false 이므로 한 번 넣은 식별자는 UPDATE 에 들어가지 않는다.[4]

급여와 수수료율

EMPLOYEES.SALARY 는 NUMBER(8,2) 이고, 검사 제약 EMP_SALARY_MIN 이 salary > 0 을 지킨다. 소수점 아래 두 자리까지의 십진수이므로 BigDecimal 로 매핑한다.

MappedEmployee 의 salary

```
@DecimalMax(value = ATTRIBUTE_DECIMAL_MAX_SALARY, inclusive = true)
@DecimalMin(value = ATTRIBUTE_DECIMAL_MIN_SALARY_EXCLUSIVE, inclusive = false)
// EMP_SALARY_MIN: CHECK (salary > 0)
@Basic(optional = true, fetch = FetchType.EAGER)
@Column(name = COLUMN_NAME_SALARY, nullable = true, insertable = true, updatable = true,
        precision = COLUMN_PRECISION_SALARY, scale = COLUMN_SCALE_SALARY,
        comment = "Monthly salary of the employee. Must be greater\n" +
                "than zero (enforced by constraint emp_salary_min)")
private BigDecimal salary;
```

ATTRIBUTE_DECIMAL_MIN_SALARY_EXCLUSIVE 의 값은 "-000000.00", 곧 0 이다. inclusive = false 이므로 @DecimalMin 은 0 보다 큰 값만 받는다. 검사 제약 salary > 0 을 그대로 옮긴 것이다. @DecimalMax 는 NUMBER(8,2) 가 담을 수 있는 가장 큰 값 999999.99 를 검사한다.

SALARY 에는 NOT NULL 이 없다. 검사 제약은 값이 NULL 이면 위반되지 않으므로 급여가 없는 직원도 넣을 수 있고, @DecimalMin 과 @DecimalMax 도 null 은 통과시킨다.

COMMISSION_PCT 는 NUMBER(2,2) 다. 소수점 아래 두 자리만 있는 수이므로 담을 수 있는 값은 -0.99 부터 0.99 까지이고, commissionPct 도 @DecimalMin 과 @DecimalMax 로 그 범위를 검사한다. 설치한 데이터의 수수료율은 0.1 부터 0.4 까지다.

날짜와 시각

EMPLOYEES.HIRE_DATE 와 JOB_HISTORY 의 START_DATE, END_DATE 는 DATE 다. 이름은 날짜이지만 LocalDate 가 아니라 LocalDateTime 으로 매핑한다([oracle-types]).

MappedEmployee 의 hireDate

```
@NotNull
@Basic(optional = false, fetch = FetchType.EAGER)
@Column(name = COLUMN_NAME_HIRE_DATE, nullable = false, insertable = true, updatable = true,
        comment = "Date when the employee started on this job. A not null column.")
private LocalDateTime hireDate;
```

설치한 데이터의 HIRE_DATE 는 모두 자정이다. LocalDate 로 받아도 지금의 데이터는 잃지 않지만, 시각이 든 값을 넣으면 시각을 잃는다.

엔티티와 식별자

엔티티는 모두 매핑된 슈퍼클래스를 상속하고 @Entity 와 @Table 을 붙인 클래스다. Region 은 MappedRegion 을 상속한다. @Table 의 comment 에는 설치 스크립트가 COMMENT ON TABLE 로 테이블에 붙인 설명을 기재했다. @Column 의 comment 에도 COMMENT ON COLUMN 으로 컬럼에 붙인 설명을 기재했다.

Region 엔티티

```
@Entity
@Table(name = MappedRegion.TABLE_NAME,
       comment = "Regions table that contains region numbers and names. references with the " +
               "Countries table.")
public class Region
        extends MappedRegion { }
```

식별자

어느 엔티티의 식별자에도 @GeneratedValue 가 없다. HR 의 시퀀스는 매핑이 쓰지 않고, 식별자의 값은 영속화하기 전에 애플리케이션이 정한다(HR (Human Resources)).

Region 의 생성자와 식별자의 setter 는 protected 이고, 정적 팩토리 메서드도 없다. 다른 엔티티도 생성자와 식별자의 setter 가 모두 protected 다. 그래서 새 인스턴스를 생성하고 식별자를 정하는 일은 같은 패키지 안이나 엔티티를 상속한 클래스에서만 할 수 있다.

NOT NULL 과 제약

NOT NULL 과 @NotNull

LOCATIONS.CITY 는 이 테이블에서 식별자 말고 NOT NULL 인 하나뿐인 컬럼이다.

MappedLocation 의 city

```
@Size(min = SIZE_MIN_CITY, max = SIZE_MAX_CITY)
@NotNull
@Basic(optional = false, fetch = FetchType.EAGER)
@Column(name = COLUMN_NAME_CITY,
        nullable = false,
        insertable = true,
        updatable = true,
        length = COLUMN_LENGTH_CITY,
        comment = "A not null column that shows city where an office, warehouse, or\n" +
                "production site of a company is located. "
)
private String city;
```

@NotNull 은 Bean Validation 이 영속화하기 전에 검사하고, optional = false 와 nullable = false 는 구현체에게 값이 늘 있다는 것을 알린다. STREET_ADDRESS, POSTAL_CODE, STATE_PROVINCE 는 NULL 을 허용하므로 @NotNull 이 없고, optional 과 nullable 이 true 다. REGIONS.REGION_NAME 도 NOT NULL 이 없으므로 @NotNull 이 없고, optional 과 nullable 이 true 다.

유일 제약

EMPLOYEES.EMAIL 은 NOT NULL 이고 유일 제약 EMP_EMAIL_UK 가 걸려 있다. 이 컬럼에는 주소 전체가 아니라 @ 앞부분만 들어 있다. 100 번 Steven King 의 것은 SKING, 101 번 Neena Yang 의 것은 NYANG 이다.

MappedEmployee 의 email

```
// TODO: 지운다. DDL 이 제약하지 않는다 -- EMPLOYEES.EMAIL 은 검사가 없는 VARCHAR2(25) 다
//    @Email
@Size(min = SIZE_MIN_EMAIL, max = SIZE_MAX_EMAIL)
@NotNull
@Basic(optional = false, fetch = FetchType.EAGER)
@Column(name = COLUMN_NAME_EMAIL,
        nullable = false,
        insertable = true,
        updatable = true,
        length = COLUMN_LENGTH_EMAIL,
        unique = true,
        comment = "Email id of the employee"
)
private String email;
```

@Column 의 unique = true 는 유일 제약을 컬럼 하나에 기재하는 줄임이다.[5] @Email 은 주석으로 남아 있다. 들어 있는 값에 @ 가 없으니 @Email 을 붙이면 설치한 데이터부터 검증을 통과하지 못하고, 테이블도 형식을 검사하지 않는다.

검사 제약

EMPLOYEES 의 검사 제약 EMP_SALARY_MIN 은 salary 의 @DecimalMin 으로 옮겼다(급여와 수수료율).

JOB_HISTORY 에서는 검사 제약 JHIST_DATE_INTERVAL 이 end_date > start_date 를 지킨다.

이 제약은 `@AssertTrue` 를 붙인 메서드로 옮겼다.

MappedJobHistory 의 isEndDateAfterStartDate

```
@AssertTrue
protected boolean isEndDateAfterStartDate() {
    if (endDate == null) {
        return true;
    }
    if (startDate == null) {
        return true;
    }
    return endDate.isAfter(startDate);
}
```

두 값 가운데 하나라도 없으면 `true` 를 돌려준다. 값이 없는 것은 `@NotNull` 이 따로 검사한다. 이 메서드에는 `@AssertTrue` 가 있으므로 Bean Validation 이 검사한다. `JobHistory` 를 영속화할 일은 없지만(트리거가 쓰는 JOB_HISTORY), 읽어 온 인스턴스를 검증할 때 쓸 수 있다.

테이블이 지키지 않는 규칙

`JOBS` 의 `MIN_SALARY` 와 `MAX_SALARY` 는 그 직무의 가장 낮은 급여와 가장 높은 급여다.

MappedJob 의 minSalary

```
@Max(ATTRIBUTE_MAX_MIN_SALARY)
@Min(ATTRIBUTE_MIN_MIN_SALARY)
@Basic(optional = true)
@Column(name = COLUMN_NAME_MIN_SALARY,
        nullable = true,
        insertable = true,
        updatable = true,
        precision = COLUMN_PRECISION_MIN_SALARY,
        scale = COLUMN_SCALE_MIN_SALARY,
        comment = "Minimum salary for a job title."
)
private Integer minSalary;
```

`NUMBER(6)` 이 담을 수 있는 `-999999` 부터 `999999` 까지를 검사한다. 급여가 음수일 수 없다는 것, 가장 낮은 급여가 가장 높은 급여를 넘을 수 없다는 것은 테이블이 지키지 않으므로 매핑도 검사하지 않는다.

`MappedJob` 에는 그런 검사를 하는 메서드가 다섯 있지만, `isEndDateAfterStartDate` 와 달리 어느 것에도 `@AssertTrue` 가 없다.

MappedJob 의 isMinSalaryPositive

```
// TODO: 지운다. DDL 이 제약하지 않는다 -- JOBS.MIN_SALARY 는 검사가 없는 NUMBER(6) 이다
//    @AssertTrue
protected boolean isMinSalaryPositive() {
    if (minSalary == null) {
        return true;
    }
    return minSalary > 0;
}
```

제약 애너테이션이 없는 메서드는 Bean Validation 이 호출하지 않으므로, 이 메서드들은 검증에 쓰이지 않는다. 그 가운데 셋은 `@AssertTrue` 를 주석 처리하고, DDL 이 제약하지 않으니 지운다는 `TODO` 를 달아 두었다. `EMAIL` 의 `@Email` 과 `COUNTRY_ID` 의 `@Size(min = …)` 도 같은 `TODO` 와 함께 주석으로 남아 있다. **테이블이 지키지 않는 규칙을 매핑이 따로 지키게 하지 않는다는 뜻이다.**

급여로 정렬하기

`JOBS` 의 `MIN_SALARY` 와 `MAX_SALARY` 는 `NULL` 을 허용한다. 그래서 급여로 직무를 정렬할 때에는 `NULL` 을 앞에 둘지 뒤에 둘지 정해야 한다. Jakarta Persistence 3.2 의 Query Language 는 `ORDER BY` 에 `NULLS FIRST` 와 `NULLS LAST` 를 기재할 수 있다. 기재하지 않으면 `NULL` 이 앞에 올지 뒤에 올지는 데이터베이스가 정한다.[6]

`MappedJob` 에는 같은 순서로 메모리에서 정렬하는 `Comparator` 가 둘 있다.

MappedJob 의 Comparator 둘

```
public static final Comparator<MappedJob> COMPARATOR_MIN_SALARY_NATURAL_NULLS_FIRST =
        Comparator.comparing(
                MappedJob::getMinSalary,
                Comparator.nullsFirst(Comparator.naturalOrder())
        );

public static final Comparator<MappedJob> COMPARATOR_MAX_SALARY_REVERSE_NULLS_LAST =
        Comparator.comparing(
                MappedJob::getMaxSalary,
                Comparator.nullsLast(Comparator.reverseOrder())
        );
```

`nullsFirst` 와 `nullsLast` 가 Query Language 의 `NULLS FIRST` 와 `NULLS LAST` 에 해당한다.

관계

외래 키의 컬럼은 매핑된 슈퍼클래스의 기본 속성이 읽고, 엔티티의 관계가 쓴다([oracle-foreign-keys]). 이 절에서는 엔티티의 관계를 살펴본다.

대일 관계

COUNTRIES.REGION_ID 는 REGIONS 를 참조하는 외래 키이고, Country 는 이 컬럼을 관계 region 으로 매핑한다.

Country 의 region

```
@Valid
@ManyToOne(optional = true, fetch = FetchType.LAZY, cascade = {})
@JoinColumn(name = COLUMN_NAME_REGION_ID, nullable = true,
            insertable = true,
            updatable = true
)
private Region region;
```

REGION_ID 에는 NOT NULL 이 없으므로, optional 과 nullable 이 true 다. @ManyToOne 의 fetch 는 기본값이 EAGER 인데, 여기서는 LAZY 로 바꿨다.[7] 나라를 읽을 때 지역은 읽지 않고, getRegion() 으로 지역에 처음 접근할 때 읽는다. HR 의 모든 @ManyToOne 이 LAZY 다.

@Valid 는 나라를 검증할 때 관계가 가리키는 지역도 함께 검증하게 한다. 대다 관계의 List<@Valid @NotNull Country> 에 붙은 @Valid 도 같은 일을 한다(대다 관계). 지역을 검증하면 그 지역의 나라들도 검증한다.

Location 의 country 는 @JoinColumn 의 nullable 에 true 대신 상수 COLUMN_NULLABLE_COUNTRY_ID 를 기재했다. 값은 true 다.

EMPLOYEES.JOB_ID 는 JOBS 를 참조하는 외래 키이고, NOT NULL 이다.

Employee 의 job

```
@Valid
@NotNull
@ManyToOne(optional = false,
           fetch = FetchType.LAZY,
           cascade = {
           }
)
@JoinColumn(name = COLUMN_NAME_JOB_ID,
            referencedColumnName = Job.COLUMN_NAME_JOB_ID,
            nullable = false,
            insertable = true,
            updatable = true
)
private Job job;
```

optional = false 이므로 직원에게는 늘 직무가 있다. referencedColumnName 은 참조하는 테이블의 컬럼 이름이다. 기재하지 않으면 참조하는 테이블의 기본 키 컬럼이 되므로 생략해도 되지만, 이 매핑은 Job.COLUMN_NAME_JOB_ID 로 기재했다.[8] department 도 같은 방법으로 DEPARTMENT_ID 를 매핑하는데, 부서가 없는 직원이 있으므로 optional = true, nullable = true 다.

대다 관계

Region 에는 컬럼이 없는 속성 countries 가 있다. 이 지역의 나라들이고, Country 의 region 이 주인인 관계의 반대쪽이다.

Region 의 countries

```
@OneToMany(
        mappedBy = Country.ATTRIBUTE_NAME_REGION,
        fetch = FetchType.LAZY,
        cascade = {
        },
        orphanRemoval = false
)
private List<@Valid @NotNull Country> countries;
```

mappedBy 는 관계의 주인이 Country 의 region 이라는 것을 가리킨다.[9] 애너테이션의 요소에는 컴파일 시점의 상수만 기재할 수 있으므로, 속성의 이름을 Country 의 상수 ATTRIBUTE_NAME_REGION 으로 기재한다. @OneToMany 쪽은 주인이 아니므로 mappedBy 를 기재하고, @ManyToOne 쪽이 주인이 된다.[10] cascade 는 비어 있고 orphanRemoval 은 false 다. 지역을 영속화하거나 지워도 그 나라들에는 아무 일도 일어나지 않는다.

HR 의 대다 관계는 모두 이와 같은 형태이고, 모두 관계의 반대쪽이다.

Table 7. HR 의 대다 관계

엔티티	속성	주인
Region	countries	Country.region
Country	locations	Location.country
Location	departments	Department.location
Department	employees	Employee.department
Department	jobHistories	JobHistory.department
Job	employees	Employee.job
Job	jobHistories	JobHistory.job
Employee	subordinates	Employee.manager
Employee	managedDepartments	Department.manager
Employee	jobHistories	JobHistory.employee

이 속성들의 getter 와 setter 는 패키지 안에서만 보인다. Location 의 getDepartments() 만 public 이다. 패키지 밖에서 지역의 나라를 구하려면 Country 쪽에서 region 으로 조회한다.

서로를 참조하는 DEPARTMENTS 와 EMPLOYEES

두 테이블이 서로를 참조한다는 것은 HR (Human Resources) 에서 보았다. **두 테이블이 서로를 참조하므로, 두 외래 키에 처음부터 서로를 참조하는 값을 넣으려 하면 어느 테이블에도 첫 행을 넣을 수 없다.**

설치 스크립트는 DEPARTMENTS 를 먼저 생성하고, EMPLOYEES 를 생성한 다음에 DEPARTMENTS 의 외래 키 DEPT_MGR_FK 를 더한다. 데이터를 넣을 때에는 DEPT_MGR_FK 를 끄고 두 테이블을 채운 뒤에 다시 켠다. 두 외래 키 컬럼이 모두 NULL 을 허용하므로, 애플리케이션이라면 한쪽을 NULL 로 넣었다가 다른 쪽을 넣은 뒤에 고치는 방법도 있다.

설치한 데이터에서 부서장은 모두 자기가 맡은 부서에서 일한다. Executive 부서(90)의 부서장은 100 번 직원 Steven King 이고, Steven King 의 부서는 90 이다. 그러나 스키마는 이것을 지키지 않는다. 부서장이 다른 부서에서 일해도 두 외래 키는

모두 만족한다.

매핑은 두 외래 키를 서로 다른 두 관계로 매핑한다.

Table 8. DEPARTMENTS 와 EMPLOYEES 사이의 관계

외래 키	주인	반대쪽	뜻
DEPARTMENTS.MANAGER_ID	Department.manager	Employee.managedDepartments	부서장과 부서장이 맡은 부서들
EMPLOYEES.DEPARTMENT_ID	Employee.department	Department.employees	직원이 일하는 부서와 부서의 직원들

Department 의 employees 에서 mappedBy 가 가리키는 것은 Employee 의 department 이고, manager 가 아니다. 그래서 Department 에서 getManager() 와 getEmployees() 는 서로 다른 것을 돌려준다. 부서장이 그 부서의 직원인지 알려면 두 관계를 함께 봐야 한다.

설치한 HR 의 부서 27 개 가운데 부서장이 있는 부서는 11 개뿐이다. 그래서 Department 의 manager 는 optional = true 이고, nullable 에는 Location 의 country 처럼 값이 true 인 상수 COLUMN_NULLABLE_MANAGER_ID 를 기재했다.

자기 자신을 참조하는 직원

EMPLOYEES.MANAGER_ID 는 EMPLOYEES 자신을 참조하는 외래 키다. 직원의 상사도 직원이다.

Employee 의 manager

```
@Valid
@ManyToOne(optional = true,
           fetch = FetchType.LAZY,
           cascade = {
           }
)
@JoinColumn(name = COLUMN_NAME_MANAGER_ID,
            referencedColumnName = COLUMN_NAME_EMPLOYEE_ID,
            nullable = true,
            insertable = true,
            updatable = true
)
private Employee manager;
```

관계의 대상이 자기 자신의 클래스일 뿐, 다른 @ManyToOne 과 다르지 않다. referencedColumnName 이 자기 테이블의 EMPLOYEE_ID 다. 사장인 100 번 Steven King 은 상사가 없으므로 optional = true 다.

반대쪽은 subordinates 다.

Employee 의 subordinates

```
@OneToMany(mappedBy = Employee.ATTRIBUTE_NAME_MANAGER,
           fetch = FetchType.LAZY,
           cascade = {
           },
           orphanRemoval = false
)
private List<@Valid @NotNull Employee> subordinates;
```

`mappedBy` 가 가리키는 것도 `Employee` 의 `manager` 다. 한 클래스 안에 관계의 주인과 반대쪽이 함께 있다.

설치한 데이터에서 Steven King 의 직속 부하 직원은 14 명이다. Steven King 부터 부하 직원을 따라 내려가면 가장 깊은 곳이 4 단계이고, 부하 직원이 있는 직원은 18 명이다.

`e.manager.manager` 처럼 관계를 이어 따라가면 두 단계 위의 상사를 구할 수 있다. **그러나 Query Language 에는 재귀(再歸, recursion) 쿼리가 없으므로, 단계의 수가 정해지지 않은 계층 전체를 한 쿼리로 따라갈 수는 없다.** SQL 로는 Oracle 의 `CONNECT BY` 나 재귀 `WITH` 로 구한다. Query Language 로는 단계마다 쿼리를 하나씩 보내거나, `subordinates` 를 메모리에서 따라간다.

트리거가 쓰는 JOB_HISTORY

`Employee` 의 `setJob` 이나 `setDepartment` 로 직무나 부서를 바꾸고 플러시하면, 트리거 `UPDATE_JOB_HISTORY` 가 바꾸기 전의 직무와 부서를 `JOB_HISTORY` 에 한 행으로 넣는다. 트리거의 정의와 주의할 점은 UPDATE_JOB_HISTORY 트리거 에서 보았다. 애플리케이션이 이 테이블에 직접 행을 넣을 일은 없다.

`EMPLOYEES` 의 다른 트리거 `SECURE_EMPLOYEES` 는 설치할 때 비활성화된다(HR (Human Resources)). `_Utils` 에는 이 트리거가 호출하는 `SECURE_DML` 과 같은 판단을 하는 메서드가 있지만, 매핑된 슈퍼클래스와 엔티티는 그 메서드를 쓰지 않는다.

설치한 `HR` 의 `JOB_HISTORY` 에는 열 행이 들어 있다.

SQL

```sql
ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY-MM-DD';
SELECT *
FROM job_history
ORDER BY employee_id, start_date;
```

결과

```
EMPLOYEE_ID START_DATE END_DATE   JOB_ID     DEPARTMENT_ID
----------- ---------- ---------- ---------- -------------
        101 2007-09-21 2011-10-27 AC_ACCOUNT           110
        101 2011-10-28 2015-03-15 AC_MGR               110
        102 2011-01-13 2016-07-24 IT_PROG               60
        114 2016-03-24 2017-12-31 ST_CLERK              50
        122 2017-01-01 2017-12-31 ST_CLERK              50
        176 2016-03-24 2016-12-31 SA_REP                80
        176 2017-01-01 2017-12-31 SA_MAN                80
        200 2005-09-17 2011-06-17 AD_ASST               90
        200 2012-07-01 2016-12-31 AC_ACCOUNT            90
        201 2014-02-17 2017-12-19 MK_REP                20
```

설치한 데이터는 트리거가 넣은 것이 아니라 설치 스크립트가 직접 넣은 것이다. 그래서 트리거가 넣었을 행과 맞지 않는 행도 있다. `176` 번 직원 Jonathon Taylor 의 입사일은 `2016-03-24` 이고 지금의 직무는 `SA_REP` 인데, 지난 직무 두 행 가운데 `SA_REP` 인 행이 바로 그날 시작한다. 열 행 가운데 일곱 행이 직원의 지금 입사일보다 뒤에 끝난다.

읽기 전용 매핑

행을 트리거가 넣으므로 `MappedJobHistory` 는 읽기 전용으로 매핑한다. 어느 속성에도 setter 가 없고, 식별자인 두 컬럼 말고는 모든 컬럼이 `insertable = false, updatable = false` 다. 식별자의 두 컬럼은 `insertable = true` 인데, EclipseLink 가 쓸 수 있는 매핑이 하나도 없는 식별자를 거부하기 때문이다. `NOT NULL` 인 `END_DATE` 가 `INSERT` 에서 빠지므로, 이 엔티티를 영속화하려 해도 데이터베이스가 받아 주지 않는다.

MappedJobHistory 의 endDate

```
@NotNull
@Basic(optional = false, fetch = FetchType.EAGER)
@Column(name = COLUMN_NAME_END_DATE,
        nullable = false,
        insertable = false,
        updatable = false,
        comment = "Last day of the employee in this job role. A not null column. Must be\n" +
                "greater than the start_date of the job_history table.\n" +
                "(enforced by constraint jhist_date_interval)"
)
private LocalDateTime endDate;
```

`EMPLOYEE_ID` 는 식별자의 일부이면서 `EMPLOYEES` 를 참조하는 외래 키다. `MappedJobHistory` 는 이 컬럼을 기본 속성 `employeeId` 로, `JobHistory` 는 관계 `employee` 로 매핑한다.

JobHistory 의 employee

```
@Valid
@NotNull
@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = COLUMN_NAME_EMPLOYEE_ID,
            referencedColumnName = Employee.COLUMN_NAME_EMPLOYEE_ID,
            nullable = COLUMN_NULLABLE_EMPLOYEE_ID,
            insertable = false, ❶
            updatable = false   ❶
)
private Employee employee;
```

❶ 같은 컬럼을 두 속성이 함께 쓰지 않도록, 관계는 읽기 전용으로 둔다.

식별자는 기본 속성 `employeeId` 가 맡는다. 그래서 Query Language 에서 `h.employeeId` 로 직원의 번호를 바로 비교할 수 있다. `employeeId` 의 getter 는 `protected` 이므로, 패키지 밖에서 직원을 읽으려면 `getEmployee()` 로 관계를 따라간다.

`job` 과 `department` 도 `employee` 처럼 `insertable = false, updatable = false` 인 `@ManyToOne` 이다. `DEPARTMENT_ID` 는 `NULL` 을 허용하므로 `department` 만 `optional = true` 다.

@IdClass 와 JobHistoryId

JOB_HISTORY 의 기본 키는 (EMPLOYEE_ID, START_DATE) 이고, JobHistory 는 @IdClass 로 이 복합 기본 키를 매핑한다.

JobHistory 엔티티

```
@Entity(name = JobHistory.ENTITY_NAME)
@IdClass(JobHistoryId.class)
@Table(name = MappedJobHistory.TABLE_NAME,
       uniqueConstraints = {
               @UniqueConstraint(
                       columnNames = {
                               JobHistory.COLUMN_NAME_EMPLOYEE_ID,
                               JobHistory.COLUMN_NAME_START_DATE
                       }
               )
       },
       comment = "Table that stores job history of the employees. If an employee\n" +
               "changes departments within the job or changes jobs within the department,\n" +
               "new rows get inserted into this table with old job information of the\n" +
               "employee. Contains a complex primary key: employee_id+start_date.\n" +
               "References with jobs, employees, and departments tables."
)
public class JobHistory
        extends MappedJobHistory { }
```

@IdClass 는 엔티티의 여러 속성으로 매핑한 복합 기본 키의 클래스를 지정한다.[11] 식별자를 이루는 속성은 매핑된 슈퍼클래스 MappedJobHistory 에 있고, 그 속성마다 @Id 를 붙인다. @Entity 의 name 은 ENTITY_NAME, 곧 "JobHistory" 이고 클래스의 이름과 같다.

@Table 의 uniqueConstraints 는 기본 키와 같은 두 컬럼을 기재한다. 기본 키가 이미 두 컬럼을 유일하게 하므로, 데이터베이스에 따로 있는 유일 제약은 아니다.

MappedJobHistory 의 startDate

```
@NotNull
@Id
@Column(name = COLUMN_NAME_START_DATE,
        nullable = false,
        insertable = true,
        updatable = false,
        comment = "A not null column in the complex primary key employee_id+start_date.\n" +
                "Must be less than the end_date of the job_history table. (enforced by\n" +
                "constraint jhist_date_interval)"
)
private LocalDateTime startDate;
```

START_DATE 도 DATE 이므로 LocalDateTime 으로 받는다(날짜와 시각). 트리거는 이 컬럼에 직원의 HIRE_DATE 를 그대로 넣는다.

JobHistoryId 는 JobHistory 의 식별자를 담는 클래스다. MappedJobHistory 의 @Id 속성과 이름도 형도 같은 두 필드를 갖고, 두 필드로 equals 와 hashCode 를 정의한다.[12]

JobHistoryId

```
public class JobHistoryId {

    @Override
    public final boolean equals(final Object obj) {
        if (this == obj) {
            return true;
        }
        if (!(obj instanceof JobHistoryId that)) {
            return false;
        }
        return Objects.equals(getEmployeeId(), that.getEmployeeId()) &&
               Objects.equals(getStartDate(), that.getStartDate());
    }

    @Override
    public final int hashCode() {
        return Objects.hash(getEmployeeId(), getStartDate());
    }

    @NotNull
    private Integer employeeId;

    @NotNull
    private LocalDateTime startDate;
}
```

JobHistoryId 는 @Embeddable 도 컬럼의 매핑도 없는 보통의 클래스다. 두 컬럼은 MappedJobHistory 의 @Id 속성이 매핑하고, JobHistoryId 는 식별자의 값을 담는 데만 쓰인다.

EntityManager 의 find 로 행 하나를 읽으려면 이 클래스의 인스턴스를 식별자로 넘긴다. JobHistoryId 의 생성자와 setter 는 모두 public 이므로, 패키지 밖에서도 인스턴스를 생성해 두 값을 정할 수 있다.

CO 스키마를 매핑한 모듈은 ORDER_ITEMS 의 복합 기본 키를 @EmbeddedId 로 매핑한다. 두 모듈은 복합 기본 키를 매핑하는 두 방법을 하나씩 보여 주고, 같은 테이블을 두 방법으로 함께 매핑하지는 않는다.

뷰 EMP_DETAILS_VIEW

EMP_DETAILS_VIEW 는 직원을 직무, 부서, 사무소, 나라, 지역과 조인해, 직원 한 사람의 정보를 한 행에 모은 뷰다.

Attribute Mappings

뷰의 정의는 아래와 같다.

EMP_DETAILS_VIEW 뷰의 정의

```
CREATE OR REPLACE VIEW emp_details_view
  (employee_id,
   job_id,
   manager_id,
   department_id,
   location_id,
   country_id,
   first_name,
   last_name,
   salary,
   commission_pct,
   department_name,
   job_title,
   city,
   state_province,
   country_name,
   region_name)
AS SELECT
  e.employee_id,
  e.job_id,
  e.manager_id,
  e.department_id,
  d.location_id,
  l.country_id,
  e.first_name,
  e.last_name,
  e.salary,
  e.commission_pct,
  d.department_name,
  j.job_title,
  l.city,
  l.state_province,
  c.country_name,
  r.region_name
FROM
  employees e,
  departments d,
  jobs j,
  locations l,
  countries c,
  regions r
WHERE e.department_id = d.department_id
  AND d.location_id = l.location_id
  AND l.country_id = c.country_id
  AND c.region_id = r.region_id
  AND j.job_id = e.job_id
WITH READ ONLY;
```

조인은 모두 내부 조인이므로, 부서가 없는 직원은 이 뷰에 나오지 않는다(HR (Human Resources)).

`WITH READ ONLY` 이므로 이 뷰로는 아무것도 쓸 수 없다. 그리고 직원에서 부서, 직무, 사무소, 나라, 지역으로 가는 조인은

모두 외래 키를 따라 많아야 한 행으로 이어진다. 그래서 이 뷰는 직원 한 사람에 한 행을 넘게 보이지 않는다.

뷰의 컬럼은 읽어 온 테이블의 NOT NULL 을 물려받는다. EMPLOYEE_ID, JOB_ID, LAST_NAME, DEPARTMENT_NAME, JOB_TITLE, CITY 가 그렇다. DEPARTMENT_ID 와 LOCATION_ID 는 내부 조인 때문에 실제로는 늘 값이 있지만, 뷰의 정의는 NOT NULL 을 기재하지 않는다.

식별자

EmpDetailsView 도 MappedEmpDetailsView 를 상속하고 @Entity 와 @Table 을 붙인 엔티티다. @Table 의 name 에 뷰의 이름을 기재하면 뷰도 테이블처럼 매핑할 수 있다.

EmpDetailsView 엔티티

```
@Entity
@Table(name = MappedEmpDetailsView.TABLE_NAME)
public class EmpDetailsView
        extends MappedEmpDetailsView { }
```

엔티티에는 식별자가 있어야 한다.[13] 뷰에는 기본 키가 없으므로, 위에서 본 대로 직원 한 사람에 한 행이라는 것을 근거로 EMPLOYEE_ID 를 식별자로 정한다.

MappedEmpDetailsView 의 employeeId

```
@Id
@Max(ATTRIBUTE_MAX_EMPLOYEE_ID)
@Min(ATTRIBUTE_MIN_EMPLOYEE_ID)
@NotNull
@Basic(optional = false)
// insertable=true 는 JPA 의 기본값이다. 굳이 기재한 것은 EclipseLink 가 @Id 의 insertable=false 를 거부하기
// 때문이다 -- "There should be one non-read-only mapping defined for the primary key field", EclipseLink-
46.
// 어차피 뷰에는 아무것도 쓰지 않는다. 다른 컬럼은 모두 insertable=false 다. 5.0.1 에서 확인했다.
@Column(name = COLUMN_NAME_EMPLOYEE_ID, nullable = false, insertable = true, updatable = false)
private Integer employeeId;
```

식별자 말고는 모든 컬럼이 insertable = false, updatable = false 이고, setter 도 없다. 식별자만 insertable = true 인 이유는 JobHistory 의 식별자와 같다(읽기 전용 매핑).

관계 없는 매핑

EmpDetailsView 는 외래 키를 관계로 매핑하지 않는다. JOB_ID, MANAGER_ID, DEPARTMENT_ID, LOCATION_ID, COUNTRY_ID 는 모두 기본 속성이다. 뷰가 직무의 이름이나 부서의 이름처럼 필요한 값을 이미 조인해 보여 주므로, 관계를 따라갈 일이 적다. 대신 길이와 범위의 상수는 원래 테이블의 매핑된 슈퍼클래스에서 가져온다. ATTRIBUTE_MIN_EMPLOYEE_ID 는 MappedEmployee.ATTRIBUTE_MIN_EMPLOYEE_ID 이고, JOB_ID 의 길이는 MappedJob 의 것이며, SALARY 의 @DecimalMin 이 쓰는 값도 MappedEmployee 의 것이다.

Exercises

이 장의 데이터로 풀 수 있는 연습문제 가운데 이 장의 매핑을 연습하기 좋은 문제를 선택해 모았다. 문제마다 Query Language 와 Criteria API 로 푼 답을 함께 실었다. 풀이는 프로젝트의 `src/test/sql-exercises` 에 있는 SQL 파일과, 같은 문제를 Query Language 와 Criteria API 로 푼 통합 테스트에 있다.

HR_SCHEMA_A_03

지금의 직무가 전에 맡았던 직무 가운데 하나인 직원을 구한다.

직원을 컬렉션 `jobHistories` 로 직무 이력과 조인하고, 이력의 직무 `h.job` 과 직원의 직무 `e.job` 을 비교한다. 두 관계 모두 `Job` 엔티티를 참조하므로 엔티티끼리 비교하면 그 식별자를 비교하는 것과 같다. 같은 직무를 두 번 이상 맡았던 직원은 이력마다 한 행씩 나오므로 `DISTINCT` 로 한 행만 남긴다.

Query Language

```
SELECT DISTINCT e
FROM Employee e
         JOIN e.jobHistories h
WHERE h.job = e.job
ORDER BY e.employeeId ASC
```

Criteria API

```
final var cb = em.getCriteriaBuilder();
final var cq = cb.createQuery(Employee.class);
final var e = cq.from(Employee.class);
final var h = e.join(Employee_.jobHistories);
cq.select(e)
        .distinct(true)
        .where(cb.equal(h.get(JobHistory_.job), e.get(Employee_.job)))
        .orderBy(cb.asc(e.get(Employee_.employeeId)));
return em.createQuery(cq)
        .getResultList();
```

Hibernate ORM 은 두 API 로 같은 SQL 을 보낸다.

Hibernate ORM 이 보내는 SQL

```
select distinct e1_0.EMPLOYEE_ID, e1_0.COMMISSION_PCT, e1_0.DEPARTMENT_ID, e1_0.EMAIL, e1_0.FIRST_NAME,
e1_0.HIRE_DATE, e1_0.JOB_ID, e1_0.LAST_NAME, e1_0.MANAGER_ID, e1_0.PHONE_NUMBER, e1_0.SALARY
from HR.EMPLOYEES e1_0
join HR.JOB_HISTORY jh1_0 on e1_0.EMPLOYEE_ID=jh1_0.EMPLOYEE_ID
where jh1_0.JOB_ID=e1_0.JOB_ID
order by e1_0.EMPLOYEE_ID
```

EclipseLink 도 두 API 로 같은 SQL 을 보낸다. Hibernate ORM 은 두 테이블의 `JOB_ID` 를 바로 비교하지만, EclipseLink 는 `JOBS` 테이블을 더 조인해 비교한다. 그 뒤에 직원의 부서, 직무, 관리자 같은 관계를 기본 키로 하나씩 읽는 SELECT 를 18개 더 보낸다. 위빙 없이 실행해서 `LAZY` 로 매핑한 다대일 관계도 즉시 읽기 때문이다.

EclipseLink 가 보내는 SQL

```
SELECT DISTINCT t1.EMPLOYEE_ID, t1.COMMISSION_PCT, t1.DEPARTMENT_ID, t1.EMAIL, t1.FIRST_NAME, t1.HIRE_DATE,
t1.JOB_ID, t1.LAST_NAME, t1.MANAGER_ID, t1.PHONE_NUMBER, t1.SALARY
FROM HR.JOB_HISTORY t0, HR.JOBS t2, HR.EMPLOYEES t1
WHERE ((t0.JOB_ID = t2.JOB_ID) AND ((t0.EMPLOYEE_ID = t1.EMPLOYEE_ID) AND (t2.JOB_ID = t1.JOB_ID)))
ORDER BY t1.EMPLOYEE_ID ASC
```

로그에 남긴 결과

```
com.github.jinahya.oracle.sample.persistence.hr.Employee@b0{employeeId=176,firstName=Jonathon,lastName=Taylo
r,email=JTAYLOR,phoneNumber=44.1632.960031,hireDate=2016-03-
24T00:00,salary=8600,commissionPct=0.2,jobId=SA_REP,managerId=149,departmentId=80}
com.github.jinahya.oracle.sample.persistence.hr.Employee@c8{employeeId=200,firstName=Jennifer,lastName=Whale
n,email=JWHALEN,phoneNumber=1.515.555.0165,hireDate=2013-09-
17T00:00,salary=4400,commissionPct=null,jobId=AD_ASST,managerId=101,departmentId=10}
```

HR_SCHEMA_A_05

위치를 받아, 그 위치의 부서와 부서마다의 직원을 쿼리 하나로 구한다. 직원이 없는 부서도 나오고, 직원은 성순, 이름순, 그다음 `EMPLOYEE_ID` 순이다. 부서를 읽은 뒤 부서마다 직원을 따로 읽으면 N+1 문제가 된다.

부서와 직원을 함께 선택하므로 결과의 원소는 `Object[]` 다. `Department` 의 `getEmployees()` 가 `public` 이 아니어서, 직원을 페치 조인한 부서 대신 부서와 직원을 행마다 선택한다. 컬렉션 `employees` 를 `LEFT JOIN` 으로 조인하므로 직원이 없는 부서는 직원이 `null` 인 행 하나로 나온다. `NULLS FIRST` 는 이름이 기록되지 않은 직원을 같은 성의 직원 가운데 앞에 둔다.

Query Language

```
SELECT d, e
FROM Department d
        LEFT JOIN d.employees e
WHERE d.location.locationId = :locationId
ORDER BY d.departmentId ASC,
        e.lastName ASC,
        e.firstName ASC NULLS FIRST,
        e.employeeId ASC
```

Criteria API

```
final var cb = em.getCriteriaBuilder();
final var cq = cb.createQuery(Object[].class);
final var d = cq.from(Department.class);
final var e = d.join(Department_.employees, JoinType.LEFT);
cq.select(cb.array(d, e))
        .where(cb.equal(d.get(Department_.location).get(Location_.locationId),
                        cb.parameter(Integer.class, "locationId")))
        .orderBy(cb.asc(d.get(Department_.departmentId)),
                 cb.asc(e.get(Employee_.lastName)),
                 cb.asc(e.get(Employee_.firstName), Nulls.FIRST),
                 cb.asc(e.get(Employee_.employeeId)));
return em.createQuery(cq)
        .setParameter("locationId", 1700)
        .getResultList();
```

두 제공자 모두 두 API 로 같은 SQL 을 보낸다.

Hibernate ORM 이 보내는 SQL

```
select d1_0.DEPARTMENT_ID, d1_0.DEPARTMENT_NAME, d1_0.LOCATION_ID, d1_0.MANAGER_ID, e1_0.EMPLOYEE_ID,
e1_0.COMMISSION_PCT, e1_0.DEPARTMENT_ID, e1_0.EMAIL, e1_0.FIRST_NAME, e1_0.HIRE_DATE, e1_0.JOB_ID,
e1_0.LAST_NAME, e1_0.MANAGER_ID, e1_0.PHONE_NUMBER, e1_0.SALARY
from HR.DEPARTMENTS d1_0
left join HR.EMPLOYEES e1_0 on d1_0.DEPARTMENT_ID=e1_0.DEPARTMENT_ID
where d1_0.LOCATION_ID=?
order by d1_0.DEPARTMENT_ID, e1_0.LAST_NAME, e1_0.FIRST_NAME asc nulls first, e1_0.EMPLOYEE_ID
```

EclipseLink 는 이 쿼리 뒤에 부서의 위치, 직원의 직무와 관리자 같은 관계를 기본 키로 하나씩 읽는 SELECT 를 19개 더 보낸다. 위빙 없이 실행해서 `LAZY` 로 매핑한 다대일 관계도 즉시 읽기 때문이다.

EclipseLink 가 보내는 SQL

```
SELECT t0.DEPARTMENT_ID, t0.DEPARTMENT_NAME, t0.LOCATION_ID, t0.MANAGER_ID, t1.EMPLOYEE_ID,
t1.COMMISSION_PCT, t1.DEPARTMENT_ID, t1.EMAIL, t1.FIRST_NAME, t1.HIRE_DATE, t1.JOB_ID, t1.LAST_NAME,
t1.MANAGER_ID, t1.PHONE_NUMBER, t1.SALARY
FROM HR.DEPARTMENTS t0
LEFT OUTER JOIN HR.EMPLOYEES t1 ON (t1.DEPARTMENT_ID = t0.DEPARTMENT_ID)
WHERE (t0.LOCATION_ID = ?)
ORDER BY t0.DEPARTMENT_ID ASC, t1.LAST_NAME ASC, t1.FIRST_NAME ASC NULLS FIRST, t1.EMPLOYEE_ID ASC
bind => [1700]
```

쿼리는 34행을 돌려준다. 아래는 Hibernate ORM 으로 실행한 로그이며, 여섯째 행의 직원은 `Employee$HibernateProxy` 이고 필드가 모두 `null` 이다. 부서 30 을 읽을 때 Hibernate ORM 이 `LAZY` 인 부서장 `manager` 를 프록시로 생성해 두었고, 같은 영속성 컨텍스트에서 같은 식별자의 직원을 그 프록시로 돌려주기 때문이다. `toString()` 은 초기화되지 않은 프록시의 필드를 그대로 읽어서 `null` 이 나오지만, 테스트가 getter 로 읽는 값은 맞다. EclipseLink 로 실행한 로그에는 모든 행의 값이 나온다.

로그에 남긴 결과

```
[com.github.jinahya.oracle.sample.persistence.hr.Department@a{id=10,departmentName=Administration,managerId=
200,locationId=1700},
com.github.jinahya.oracle.sample.persistence.hr.Employee@c8{employeeId=200,firstName=Jennifer,lastName=Whale
n,email=JWHALEN,phoneNumber=1.515.555.0165,hireDate=2013-09-
17T00:00,salary=4400,commissionPct=null,jobId=AD_ASST,managerId=101,departmentId=10}]
[com.github.jinahya.oracle.sample.persistence.hr.Department@1e{id=30,departmentName=Purchasing,managerId=114
,locationId=1700},
com.github.jinahya.oracle.sample.persistence.hr.Employee@74{employeeId=116,firstName=Shelli,lastName=Baida,e
mail=SBAIDA,phoneNumber=1.515.555.0116,hireDate=2015-12-
24T00:00,salary=2900,commissionPct=null,jobId=PU_CLERK,managerId=114,departmentId=30}]
[com.github.jinahya.oracle.sample.persistence.hr.Department@1e{id=30,departmentName=Purchasing,managerId=114
,locationId=1700},
com.github.jinahya.oracle.sample.persistence.hr.Employee@77{employeeId=119,firstName=Karen,lastName=Colmenar
es,email=KCOLMENA,phoneNumber=1.515.555.0119,hireDate=2017-08-
10T00:00,salary=2500,commissionPct=null,jobId=PU_CLERK,managerId=114,departmentId=30}]
[com.github.jinahya.oracle.sample.persistence.hr.Department@1e{id=30,departmentName=Purchasing,managerId=114
,locationId=1700},
com.github.jinahya.oracle.sample.persistence.hr.Employee@76{employeeId=118,firstName=Guy,lastName=Himuro,ema
il=GHIMURO,phoneNumber=1.515.555.0118,hireDate=2016-11-
15T00:00,salary=2600,commissionPct=null,jobId=PU_CLERK,managerId=114,departmentId=30}]
[com.github.jinahya.oracle.sample.persistence.hr.Department@1e{id=30,departmentName=Purchasing,managerId=114
,locationId=1700},
com.github.jinahya.oracle.sample.persistence.hr.Employee@73{employeeId=115,firstName=Alexander,lastName=Khoo
,email=AKHOO,phoneNumber=1.515.555.0115,hireDate=2013-05-
18T00:00,salary=3100,commissionPct=null,jobId=PU_CLERK,managerId=114,departmentId=30}]
[com.github.jinahya.oracle.sample.persistence.hr.Department@1e{id=30,departmentName=Purchasing,managerId=114
,locationId=1700},
com.github.jinahya.oracle.sample.persistence.hr.Employee$HibernateProxy@72{employeeId=null,firstName=null,la
stName=null,email=null,phoneNumber=null,hireDate=null,salary=null,commissionPct=null,jobId=null,managerId=nu
ll,departmentId=null}]
...
[com.github.jinahya.oracle.sample.persistence.hr.Department@10e{id=270,departmentName=Payroll,managerId=null
,locationId=1700}, null]
```

[1] Oracle. Human Resources Sample Schema. https://github.com/oracle-samples/db-sample-schemas/tree/main/human_resources

[2] Oracle. Oracle Database SQL Language Reference 23ai, Nulls. "The database treats a character value with a length of zero as null." https://docs.oracle.com/en/database/oracle/oracle-database/23/sqlrf/Nulls.html

[3] Jakarta Persistence 3.2, 11.1.9 Column Annotation. "(Optional) The precision of a column of SQL type decimal or numeric, or of similar database-native type. Applies only to columns of exact numeric type." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a14330

[4] Jakarta Persistence 3.2, 11.1.9 Column Annotation. "(Optional) Whether the column is included in SQL UPDATE statements generated by the persistence provider." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a14330

[5] Jakarta Persistence 3.2, 11.1.9 Column Annotation. "(Optional) Whether the column is a unique key. This is a shortcut for the UniqueConstraint annotation at the table level and is useful for when the unique key constraint corresponds to only a single column." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a14330

[6] Jakarta Persistence 3.2, 4.10 ORDER BY Clause. "If NULLS is not specified, the database determines whether null values occur first or last." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a5587

[7] Jakarta Persistence 3.2, 11.1.31 ManyToOne Annotation. "FetchType fetch() default EAGER;" "The LAZY strategy is a hint to the persistence provider runtime that the associated entity should be fetched lazily when it is first accessed." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#manytoone-annotation

[8] Jakarta Persistence 3.2, 11.1.26 JoinColumn Annotation. "(Optional) The name of the column referenced by this foreign key column." "(Default only applies if single join column is being used.) The same name as the primary key column of the referenced table." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a14922

[9] Jakarta Persistence 3.2, 2.11 Entity Relationships. "The inverse side of a bidirectional relationship must refer to its owning side by use of the mappedBy element of the OneToOne, OneToMany, or ManyToMany annotation." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a516

[10] Jakarta Persistence 3.2, 2.11 Entity Relationships. "The many side of one-to-many / many-to-one bidirectional relationships must be the owning side, hence the mappedBy element cannot be specified on the ManyToOne annotation." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a516

[11] Jakarta Persistence 3.2, 11.1.23 IdClass Annotation. "The IdClass annotation is applied to an entity class or a mapped superclass to specify a composite primary key class that is mapped to multiple fields or properties of the entity." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a14836

[12] Jakarta Persistence 3.2, 2.4.1 Composite primary keys. "If the composite primary key class is represented as an id class, the names of primary key fields or properties of the primary key class and those of the entity class to which the id class is mapped must correspond and their types must be the same." "The primary key class must define equals and hashCode methods." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#composite-primary-keys

[13] Jakarta Persistence 3.2, 2.4 Primary Keys and Entity Identity. "Every entity must have a primary key." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a132